

An accuracy–leniency tension in LLM-judge metrics

Seyi Akadri*

Edesiri Bridget Nkemnole†

September 2026

Abstract

LLM judges are increasingly used not only to measure a system's behaviour but as the objective that system is improved against. Once a metric becomes a target, the question is no longer whether it is accurate but whether a movement in it remains evidence about capability. We study that question in a setting where the answer is checkable: a task family with a complete verifier, and a learner whose objective ability is held static by construction.

Two preregistered experiments establish the apparatus. The first is a validity demonstration rather than a discovery — because ability cannot move, a rise in the logged metric under self-selection is expected, and we report it (+0.33834, 86/86 replicates positive) as evidence that the instrument detects exposure selection well above the smallest effect the design was sized to find. The second shows that evaluator exploitation is measurable apart from a known, deliberately distorted evaluator (+0.06600, one-sided lower bound +0.04645).

Carrying the apparatus to a real LLM judge produced a boundary rather than an effect. Without a field in which to reason, the judge agreed with the verifier on 61.00% of items while approving 26.60% of work the verifier had rejected; given one, it reached 99.78% but approved only 0.20%. A human scored 50 of 50 on the same items, so the inaccurate regime reflects difficulty rather than ambiguity. At the leniency that remains, the preregistered primary would need about 42,311 calls for a single replicate pair against a 45,000-call budget. **No confirmatory primary was run, and we report no effect.** We measured two operating points, not a curve, and claim no frontier: what we found is that judge competence and measurable leniency did not coexist at an affordable operating point, and that the apparatus needed to establish this is itself the transferable result.

1 Introduction

An LLM judge begins life as a measurement. It is asked how good an output is, its answer is recorded, and nothing depends on the answer except the record. It does not stay there. Once the judge's score is tracked over time, compared across versions, or used to choose between candidates, it becomes an objective — and a system improved against an objective will improve against *that objective*, which is not the same thing as improving.

This is not a claim that LLM judges are bad, and the framing matters because the two questions are easy to conflate. A judge can be accurate, well calibrated and carefully built, and still stop carrying information about capability the moment it becomes the thing being optimized. The question is not the judge's quality but its **evaluability**: under what conditions does a change in the metric remain evidence about a change in the thing the metric is for?

*Geomobile Research Nigeria. Corresponding author.

†Department of Statistics, University of Lagos, Lagos, Nigeria.

1.1 Why the question is hard to study

Two things are needed at once, and they pull apart. The setting must have a **known truth**, or there is nothing against which to say the metric drifted; and the evaluator must be **real**, or the result says nothing about the instruments practitioners actually deploy. Synthetic evaluators give the first and forfeit the second. Field studies give the second and forfeit the first.

We take the two in sequence rather than together. A task family with a complete verifier supplies known truth, and capability is pinned flat by construction so that any movement in a logged metric is attributable to what the policy selected rather than to what the learner became. Within that setting we separate two ways a metric can rise without capability rising — narrowing the distribution of attempted tasks, and exploiting the evaluator's imperfections — and validate that the apparatus detects each. Only then do we replace the known evaluator with a real LLM judge and ask what the apparatus measures.

1.2 Contributions

1. **An apparatus that separates narrowing from evaluator exploitation** under known truth, preregistered, with controls that fail loudly rather than quietly (§3).
2. **A negative result about LLM-judge metrics**: across the two operating points we tested, judge accuracy and exploitable leniency moved in opposition, and no configuration satisfied both the accuracy a credible metric needs and the leniency a measurable effect needs (§5).
3. **An infrastructure finding**: a widely used evaluation framework cannot supply the archival provenance a longitudinal experiment of this kind requires, for reasons that are structural rather than incidental (§6).

1.3 What this paper does not claim

The learner in §3 is a predictor and a selection policy over synthetic tasks, not an LLM agent; the only real LLM anywhere in this work is the judge in §5. Because ability is static by construction, nothing here shows narrowing *masquerading as* genuine capability gain, which would require a setting where capability can move. We ran no confirmatory primary against the real judge and report no effect. And two operating points are two points: we describe a tension we measured, not a frontier we mapped. §8 states these limits in full rather than leaving them to be inferred.

2 Background

2.1 Evaluation becomes a data-collection problem under adaptation

Offline evaluation is only as informative as the data-collection process that produced the log. In recommender and contextual-bandit settings, off-policy estimators use logged actions, propensities and outcomes to estimate the value of a target policy, with overlap and variance controlling what can be learned from the data [7–9]. Feedback loops make the issue more direct: the system changes what is shown, and the resulting observations become the material from which later models and metrics are computed [1, 5]. A metric can then move because the observed distribution moved, even when the underlying ability or objective has not.

This literature supplies the motivation for treating evaluability as a property of the whole measurement process. It does not, by itself, provide a real external judge with deterministic task truth against which evaluator errors can be measured. That is the gap the apparatus in §3 addresses.

2.2 LLM judges are scalable, but their agreement is conditional

LLM-as-a-judge methods use a language model to score or compare generated outputs when direct human evaluation is expensive. MT-Bench and Chatbot Arena made the case that a strong model can agree with human preferences at useful levels while also documenting position, verbosity, self-enhancement and reasoning limitations [10]. Open evaluator models such as Prometheus and Prometheus 2 make the evaluator more controllable and support custom rubrics or pairwise assessment [3, 4].

Those results establish the practical value of LLM judges and the importance of calibration against human judgments. They do not establish that a judge's errors are harmless when the evaluated system is improved against the judge's score. A high average agreement rate is not the same quantity as the conditional probability that a judge approves a verifier-failing output under an adaptive selection policy.

2.3 Bias and reasoning are design variables, not guarantees

Subsequent work has treated judge bias as an empirical object, measuring systematic sensitivity to properties of the response or of the evaluation context rather than treating the score as a transparent readout [2]. Work on reasoning-enabled judges similarly treats planning or intermediate reasoning as a way to improve evaluation quality, not as a guarantee that the final decision is valid for every task family [6]. These directions motivate the two AG2b operating points: a judge without enough working space and a judge with enough working space are distinct instruments whose accuracy and error surfaces must be measured.

The distinction matters for the present study. AG2b does not score the free text of a rationale, and it does not claim that a visible rationale is true or faithful. The binary verdict remains the metric; the reasoning field is working space and archived evidence. The question is whether adding that working space changes the judge's agreement with a complete verifier and leaves a measurable error surface for an adaptive experiment.

2.4 What this paper adds

Existing LLM-judge studies establish scalable judging, document systematic biases, or improve evaluator agreement. Existing recommender and off-policy work establishes that adaptive data collection changes what can be inferred from logs. AG2b joins these concerns in a deliberately bounded setting: a real external LLM judge receives executable tasks with complete deterministic truth, an independent human checks that the truth is readable, and adequacy gates are fixed before the paid screen and pilot.

The result is not a general law about LLM judges. It is a measurement-boundary result for one judge model, one rubric, one task family and the tested output budgets. The apparatus finds one operating point that is too inaccurate to be a credible metric and another that is accurate but has almost no measurable leniency. Sections 4 and 5 test that boundary; §8 states the limits.

3 Apparatus

Before asking whether a real judge's metric survives being optimized against, we need a setting where the answer is checkable. This section describes one, and reports what it establishes. It is methods rather than contribution: the results here are largely guaranteed by the construction, and we say where.

3.1 A task family with an exact verifier

Each task is a triple of an initial state, a target state and a generating program. States are eight integer registers holding values 0–9; programs are sequences of **set** and **add** operations, the latter modulo 10. The difficulty axis is program length $d \in \{1, \dots, 8\}$, fixed a priori rather than tuned.

Two properties matter. The verifier is **complete**: it executes a candidate program and compares the resulting state to the target, so verifier agreement *is* task truth rather than a proxy for it. And the features $\phi(\mathbf{x})$ visible to a selection policy are a noisy, partial view of difficulty, computable from the initial and target states alone — so selection is learnable but imperfect, which is the condition under which the question is interesting.

Objective ability is **static by construction**. The success contract is $P(\text{success} \mid \mathbf{x}) = \sigma((\mathbf{a} - \mathbf{d})/\mathbf{s})$ with $\mathbf{a}$ fixed, and retraining changes which tasks are attempted but never $\mathbf{a}$. Every result below therefore sits over a capability floor that cannot move.

3.2 Selection alone inflates the logged metric

The learner is a regularized logistic predictor of success over $\phi(\mathbf{x})$, paired with a ρ -mix selection policy: with probability ρ it selects the task it predicts it will pass, otherwise it selects uniformly. Arms differ in ρ and in nothing else, under common random numbers.

Figure 1 shows the logged policy component across the ρ -grid over 86 replicates. The preregistered contrast between the extreme arms is $H2 = +0.33834$ (95% CI $+0.32406$ to $+0.35262$), $t(85) = +47.10$, with 86/86 replicates positive.

This is a validity demonstration, not a discovery. Because ability is static, a large effect here is *expected*; it is evidence that the apparatus detects exposure selection, at 6.77 times the smallest effect it was sized to find, and nothing more. In particular it does not show that adaptive systems mislead evaluators in the wild, and it cannot show narrowing masquerading as genuine capability gain — that needs a setting where capability can move, which this one deliberately is not.

3.3 Exploitation is measurable apart from a known distortion

The second stage adds exactly one new source of non-identifiability: an imperfect evaluator. The loop optimizes a deliberately distorted signal $J(\mathbf{x}) = \alpha \cdot T(\mathbf{x}) + \beta \cdot \mathbf{z}(\mathbf{x}) + \epsilon$ while true outcomes T are retained on the logged path, so the gap between what the evaluator rewards and what is true can be read directly. Identifiability requires $\mathbf{z}$ to be learnable from $\phi(\mathbf{x})$ — or exploitation is impossible by construction — and orthogonal to difficulty — or it is aliased with narrowing. Both are asserted numerically in an adequacy gate rather than assumed.

Figure 2 shows both channels across the distortion grid over 20 replicates. At the preregistered $\beta^* = 0.5$ the exploitation statistic is $+0.06600$ with a one-sided 95% lower bound of $+0.04645$, $t(19) = +5.840$, and 19/20 replicates positive.

Two cautions travel with that figure. First, **ex and narrowing are not a variance decomposition**. Their sum is an accounting identity, and a policy can narrow specifically toward evaluator-optimistic regions, so the two interact; Figure 2 plots them as separate series and never stacks them. Second, what is established is that exploitation is *measurable* under these conditions, not that the two mechanisms are cleanly separable. Both conditions — a known distortion and retained true outcomes — fail for a real evaluator, which is precisely what §5 goes on to test.

A descriptive observation worth recording without inferential claim: measured narrowing was lower at $\beta = 1$ than at $\beta = 0$ in 20 of 20 replicates. That is consistent with a finite selection budget, in which pressure spent on exploitation is unavailable for narrowing.

3.4 Controls that fail loudly

Three design choices do more work than their size suggests. The policy-independent channels are flat by construction, so a movement in them indicates an implementation fault rather than a finding, and the analysis raises before reporting a primary if that control fails. Arms share every seed role and differ only in the policy parameter, so the contrast is paired rather than merely matched. And each stage's gates are evaluated *before* its confirmatory statistic is computed, in code rather than in prose.

None of this makes the apparatus right. It makes the ways it could be wrong noisy rather than silent, which is the property §5 depends on when it asks a question whose answer we could not check independently.

4 Carrying the apparatus to a real judge

Replacing the known evaluator with a real one removes both conditions that made §3.3 interpretable: the distortion is no longer known, and there is no second channel carrying truth alongside the judge's opinion. What remains is the deterministic verifier, which now plays a different role — not a component of the evaluator, but an independent reference against which the judge can be scored.

4.1 The event, fixed before the data

The confirmatory event is the conjunction *judge approves* and *verifier rejects*, counted over all scheduled eligible trajectories under a judge-directed selection policy and compared against a contemporaneous truth-directed control. Counting a conjunction rather than comparing a continuous judge score to a boolean verdict avoids needing a calibration map between the two scales, which is not identified when the judge's scale may itself depend on the item.

The judge is a fixed model at temperature zero with a frozen rubric, a strict response schema, and enforced token bounds. Every call is archived with its prompt text and hash, the raw response bytes, the provider's envelope fields, and a hash chain over the ledger, so that what was sent and what came back are recoverable rather than reconstructed.

4.2 Why an adequacy gate precedes the power calculation

The obvious next step after fixing an event is to size the experiment for it. Doing that first is a mistake, and it is the mistake this work initially made: we planned against a 5% base rate that later measurement put two orders of magnitude away in one configuration and one order in the other.

A power calculation answers *how much data do I need to detect an effect of this size*. It presupposes that the instrument can exhibit the effect at all. An adequacy gate asks that prior question, and it has to be answered with measurements rather than with expectations, because the quantity it turns on — how often a real judge wrongly approves work a verifier rejects — is not something a designer can introspect.

We therefore fixed eight adequacy conditions before running the experiment they gate. Three are static and cost nothing: the judge must not be shown the answer, the trajectories must be real and executable, and their truth must be recoverable by a competent human. Five require measurement: the judge must be lenient enough for the event to occur, the resulting event rate must be affordable, the judge must be accurate enough to be credible, its leniency must vary across failure modes so

that a selection policy has structure to exploit, and its verdicts must be reproducible on identical input.

4.3 The condition that does the work

Two of those conditions are deliberately opposed. The judge must be **wrong often enough to study** and **right often enough to be credible**, and a single threshold cannot be satisfied by moving in one direction.

That opposition is the part we would transfer to another experiment. Without it, a task that had simply become vague would pass an adequacy gate: ambiguity raises the error rate, and an error rate is what a leniency floor measures. With it, vagueness fails the accuracy condition while genuine difficulty need not. The human readability check in §5.3 exists to close the remaining gap, since a task can be both unambiguous to a careful reader and beyond the judge.

Neither condition was chosen to be passed. Both were fixed, with their thresholds, before the runs that would be measured against them.

5 Results

We report what a real LLM judge did on a task whose truth is known exactly, at two operating points implementing one intervention: whether the judge had working space to compute before committing to a verdict.

5.1 A judge with no room to reason

At the first operating point the judge's response schema carried a single field, a binary verdict, with `additionalProperties` false, under a 128-token output bound. Over 900 conditional units it agreed with the deterministic verifier on 61.00% (95% Wilson CI 57.77–64.13), against a preregistered adequacy floor of 80%. Its leniency — the rate at which it approved work the verifier had rejected — was 26.60% (22.92–30.64).

The immediate reading is that the judge is weak. The archive says something more specific. **Every one of the 1,075 calls returned exactly six completion tokens**, with `finish_reason` `stop` on all of them. Six tokens is `{"verdict": "FAIL"}` and nothing else. The judge was asked to trace a program of up to ten modular-arithmetic operations and emit a verdict with no intermediate state, and the schema gave it nowhere to put any. It was not performing badly at the task so much as being prevented from attempting it.

Two further measurements support that reading rather than the weak-judge one. Accuracy was close to flat in program length — 70.2% at `d=1` falling only to 56.2% at `d=8` — where a judge defeated by length would degrade steeply. And the judge was perfectly self-consistent: over five identical rescores of each of 35 items, no item ever returned two different verdicts. At temperature zero its 39% error rate was systematic, not noise. A forced choice made without computation is wrong reliably.

One asymmetry in that regime is worth recording because it bears on the rubric rather than on the judge's competence. Of the correct programs, the judge approved 64.5% of minimal ones but only 26.5% of correct-but-longer ones, which contributed 147 of the 218 false rejections. The rubric instructs the judge in terms not to reward brevity. It did so anyway.

5.2 A judge given room to reason

At the second operating point we gave the judge somewhere to compute. This was one intervention with three inseparable parts: the response schema gained a **reasoning** field declared before the verdict, the rubric gained an instruction to use it, and the output bound rose from 128 to 768 tokens. They cannot be varied independently: a field the rubric never mentions is unlikely to be used, an instruction to trace with nowhere to write cannot be followed, and either within the original bound truncates. That last point is measured rather than assumed: an intermediate 384-token attempt returned **finish_reason: length** on every unit of its pre-flight canary and produced no verdict at all.

What did not change was the task: the family, constructions, deterministic verifier, and rubric's grading instruction — the sentence defining what counts as success — were held fixed, and the schema's verdict field is identical. The comparison therefore attributes the change to the presence of working space, and **cannot attribute it to any one of the three parts alone**. Separating them would require configurations that truncate by construction, which is why we did not attempt it.

Over 900 conditional units the judge agreed with the verifier on **99.78%** (99.19–99.94) — two errors in nine hundred items. Its leniency fell to **0.20%** (0.04–1.12). An independent 300-unit screen, run first under the same configuration, measured 99.67% and 0.67%; two samples of different size landing 0.11 points apart is the reproducibility claim the near-zero leniency needs, and it is cheap to make.

Maximum observed completions were 562 on the screen and 581 on the pilot, so the 768-token bound held with room, and 512 would not have.

The completion distribution shows why an intermediate output bound is not a third operating point on this axis. Counting the 1,075 pilot responses retrospectively, the share with a completion at or above each bound was:

requested bound	responses at or above bound	inferred truncation rate
128	964/1,075	89.7%
256	576/1,075	53.6%
320	389/1,075	36.2%
384	218/1,075	20.3%
448	94/1,075	8.7%
512	23/1,075	2.1%
640	0/1,075	0.0%
768	0/1,075	0.0%

These are inferred truncation rates, not measurements from rerunning the judge at each bound. Under the preregistered five-percent unresolved-unit rule, bounds through 448 would not yield a reportable operating point. A 512-token run would be viable by this diagnostic, but it was not measured; the approximately two percent of longest traces it would exclude are the hardest items, so treating it as equivalent to 768 would bias accuracy upward. The available evidence therefore supports a threshold between no usable working space and enough space, rather than a smooth interpolation between the two measured regimes.

5.3 The task is not ambiguous

A judge that is 61% accurate on a task nobody can do tells us about the task. So before treating the first operating point as evidence about judges, we checked whether the truth was recoverable at

all. A competent human adjudicated a blind 50-item sample drawn from the same constructions, seeing the initial state, the target state and the program, and nothing else. **They agreed with the deterministic verifier on 50 of 50.** The verdicts were committed to version control before the score was computed, which is what makes the ordering evidence rather than assertion.

The task generator and the renderer were byte-identical between that adjudication and both runs, so the items a human read are the items the judge was sent. (The rubric changed between the two runs, as §5.2 describes; the adjudicator never saw it, so it does not bear on this comparison.) The inaccurate regime is therefore difficulty, not vagueness — the distinction on which everything downstream depends.

5.4 What the remaining leniency costs

The preregistered primary counts judge-approved verifier failures under a judge-directed policy against a contemporaneous control. Its power therefore depends on how often that joint event occurs at all. Using the pilot's observed balanced-law joint event rate of 0.111% as a feasibility base, detecting a doubling under the exploratory independent two-proportion approximation requires about **42,311 calls for a single pair** — roughly 94% of the entire 45,000-call primary and panel allocation. The pilot schedule did not identify the control-policy (`p_0`); this rate is a sizing input, not an estimate of it.

That figure is optimistic twice over. It treats trajectories within an arm-run as independent when they are not, so it is a lower bound; and the preregistered analysis takes the replicate pair as the unit of independence, so several pairs are required rather than one. The experiment is not merely underpowered at this operating point. It does not fit.

5.5 No admissible operating point was found

Figure 3 places both measurements in the plane of judge accuracy against judge leniency, with the two preregistered adequacy floors drawn as axes. A usable operating point must lie in the upper-right quadrant: accurate enough for the metric to be credible, and lenient enough for the effect to be measurable. Both measured points lie outside it, in opposite corners. The quadrant is empty.

We measured two points, not a curve, and we do not claim a frontier. What we claim is narrower and, we think, harder to dismiss: the intervention that fixed the accuracy failure removed almost all of the leniency, and no configuration we tested satisfied both requirements at once.

6 An infrastructure finding

A longitudinal claim about an evaluator requires that the evaluator's behaviour at two points in time be comparable, which in turn requires that what it was asked and what it answered be recoverable at both. We reviewed a widely used open-source LLM-evaluation framework, DeepEval 4.2.2, against that requirement. The structural findings below apply to this reviewed release unless they are independently rechecked against a later version.

The findings below concern that release. We also patched them in a fork (github.com/seyi/deepeval, commit `dfa3eac6029ae6052b1399123799ac252566206a`) to establish how much of the gap is closable without redesigning the framework, and the answer is instructive: less than it appears.

The fork records each judge prompt and result on the shared `generate_with_schema_and_extract` path, and on GEval's raw scoring path, and forwards both into the metric data. We measured what that captures on live traffic in a small preregistered pilot. `TaskCompletionMetric` and GEval ran on 20 traces, three repeats each, against `gpt-4.1-mini-2025-04-14`, for 242 calls. A recording transport on the provider client served as ground truth, holding the exact bytes sent and received. The preregistered rule counts a field as captured only if every call of that type was captured and the field was exact in every matched pair.

What the fork captures, it captures well: every judge call (240 of 240), and every prompt byte-identical to the one sent (240 of 240). **But what is retained for a structured response is `str(result)`, not the result.** A schema object is stored as the rendering `verdict=1.0 reason='...'`. It matched the response on the wire in 0 of 180 structured calls and did not parse back into its schema in any of them. Only GEval's scoring call, which DeepEval makes without a schema, was captured exactly (60 of 60). No provider metadata was captured on any call: no request identifier, served model, finish reason, token usage, headers or raw body. GEval's reported score could be recomputed from the capture in only 39 of 60 evaluations. The capture is not corrupt; it is incomplete in one named way. GEval reports a logprob-weighted average of the candidate scores, and the fork does not capture logprobs.

On replay the fork makes matters worse, not better. On a cache hit, including the hit after the rubric had been changed, the returned metric data carried the cold run's captured prompts and responses unchanged. The only field that differed was the evaluation cost. Before the fork, a replayed result merely looked live. With the fork, it also carries what reads as evidence of a live call: prompt and response bytes from an earlier exchange, under a rubric that is no longer current. The direct remedies are to disable the cache whenever capture is enabled, or to mark replayed captures as replays.

The other gaps are unchanged by the fork: nothing is written to durable storage until a caller serializes it, and the cache is still blind to rubric and version and still silent about whether a result was live or replayed. A judging layer built on this framework would therefore still need to own its archival contract: serializing and hashing the exact prompt, response, configuration, request metadata and live/replay status. That is what the apparatus of Section 4 does, without a framework dependency.

A later fork commit implements these remedies. After the pilot reported, commit `b651f7f80ba53e51f57d1161d22e3fb66d9500b1` on the fork's `judge-provenance-replay` branch adds what the measurements above found missing: the HTTP entity body with its SHA-256 rather than a rendering of the parsed object, the logprobs that GEval's score depends on, provider metadata, and a `live/replay` marker set on cache reads, with rubric- and version-sensitive cache keying behind an opt-in flag that is off by default. **We report this for provenance, not as a result.** The numbers in this section were measured against `dfa3eac` and describe that commit. The later commit has not been put through the pilot, so whether it reaches the acceptance criteria — structured responses round-tripping, GEval scores recomputing, a warm mutated step that misses the cache or returns captures marked as replays — is untested here and remains future work.

In the reviewed release, three properties matter and are absent. The framework's metric path retains the extracted verdict and its stated reason but **discards the judge's raw input and output**, so the bytes that would let a later reader confirm what was asked are gone. There is **no metadata distinguishing a live call from a cached replay**: a cache hit is substituted for a result and is indistinguishable downstream. And neither the cache key nor the configuration comparison performed on lookup includes **the package version or the prompt-template text**,

so an upgrade that changes a rubric returns a score computed under the previous rubric as though it were current. The comparison does check the evaluator's model identifier, so a change of judge model correctly misses the cache; the gap is specific to the rubric and the version, not general.

The third property is the one that matters most here, and we verified it by execution rather than by reading the source: with a stubbed judge counting its own invocations, a cold cache produced calls, a warm cache produced none, and a warm cache whose rubric had been mutated also produced none. A changed rubric hit the cache. The live pilot reproduced this against a real judge: 2, 0 and 0 provider calls for the same three steps.

For a longitudinal experiment this silently merges two of the three things such an experiment must keep apart — instrument noise, implementation drift, and the change being measured. An upgrade mid-experiment presents as *no change*, which is worse than presenting as noise, because noise is visible.

What this is not. It is not a claim that the framework is defective. The conditions it fails are ones no regression-testing framework advertises meeting, and it is built to tell a practitioner whether today's outputs have regressed against yesterday's expectations — a job for which discarding raw I/O and caching aggressively are reasonable choices. The finding is about what longitudinal evaluation requires, and the consequence we drew from it was to own the judge call and the archival layer directly rather than to ask the framework for something it does not offer.

One consequence is worth stating because it constrains this paper's own claims: where our other experiments claim byte-identical regeneration, no result involving an external judge can. The judge cannot be reconstructed, only replayed from the archived ledger, and we name that weaker property rather than letting the stronger one be assumed.

7 Discussion

7.1 What the apparatus establishes

The first experiment establishes that the instrument works, and little more. With objective ability pinned flat, a self-selecting policy still moved the logged metric by $+0.33834$ with every one of 86 replicates positive. That is the expected outcome of the construction, and its value is diagnostic: a measurement apparatus that failed to detect exposure selection under these conditions would be unusable for the harder question, and this one detects it comfortably above the floor the power analysis set. Read as a finding about adaptive systems it would be circular. Read as a calibration of the instrument it is what licenses everything after it.

7.2 What adding a known distortion establishes

The second experiment adds exactly one source of non-identifiability — an evaluator that is wrong in a known direction — and shows that the resulting exploitation is measurable apart from distribution narrowing: $+0.06600$, with a one-sided lower bound of $+0.04645$ excluding zero.

The word doing work there is *measurable*. Their sum is an accounting identity, not a variance decomposition, and a policy can narrow specifically toward evaluator-optimistic regions, so the two interact rather than partitioning. What the experiment shows is that an exploitation channel can be read off at all, and it shows this under two conditions that a real evaluator does not satisfy: the distortion is known, and true outcomes are retained alongside the evaluator's. Removing either is what §5 does.

7.3 What the real judge rules out

The third experiment was designed to ask whether optimization pressure degrades a real judge's metric. It did not answer that question. What it established instead is a constraint on asking it.

Without a field in which to compute, the judge was inaccurate and lenient; given one, it became accurate and almost perfectly strict. The intervention that fixed the accuracy failure removed the leniency the experiment needed. What this rules out is narrower than a general claim about LLM judges: at the two operating points we tested, with this judge, this task family and this rubric, no configuration satisfied both an accuracy floor a credible metric requires and a leniency floor a measurable effect requires. At the leniency that survived, one replicate pair would consume about 42,311 calls of a 45,000-call budget, before the clustering that makes independent sizing a lower bound and before the several pairs the analysis requires.

The human adjudication is what makes this a statement about the judge rather than about the task. A reader is entitled to suspect that a judge scoring 61.00% is being asked something ill-posed. A competent human scored 50 of 50 on the same items, seeing exactly what the judge saw.

7.4 Why the result is bounded rather than universal

We measured two operating points. Two points establish that a tension occurred; they do not establish a curve, a frontier, or a trade-off law, and we resist all three words. On the output-bound axis, the pilot's completion distribution does not offer a clean intermediate point: bounds through 448 would exceed the unresolved-unit rule, while 512 is only an unmeasured, selectively truncated case. A different judge or task family could still occupy the admissible region of Figure 3, but that would test generality rather than interpolate this instrument.

What survives the boundedness is the shape of the difficulty. A judge's errors are what make it exploitable. If that relationship holds generally, then accuracy and exploitability are not independent design targets, and an evaluator cannot be made simultaneously more trustworthy and more informative about optimization pressure by the same intervention. We observed that relationship twice. We are not in a position to assert it.

7.5 Implications for evaluator design

Three of these are practical and follow from what we did rather than from what we found.

An adequacy gate belongs before a power calculation. We sized an experiment against a 5% base rate that turned out to be off by two orders of magnitude in one configuration and one in the other. The gate that caught this had two deliberately opposed conditions — the judge must be wrong often enough to study and right often enough to be credible — so a task could not pass by becoming vague. That opposition is transferable to any experiment that optimizes against a learned evaluator.

A low judge accuracy is not self-interpreting. It is consistent with a hard task, an ambiguous task, and a hobbled judge, and these have different remedies. A cheap human readability check distinguishes them, and without it the headline comparison in §5 would have been uninterpretable.

Archival provenance is a design requirement, not an operational nicety. §6 records an evaluation framework that discards the judge's raw input and output and whose cache is blind to both package version and rubric text. For a longitudinal question that silently merges instrument drift with the change being measured.

7.6 A methodological remark on verifying claims

The numerical claims in this paper are checked mechanically: a tool re-derives every reported figure from the committed artifacts and fails if the prose and the data disagree. We report what happened when we tested that tool adversarially, because the result was not what inspection suggested.

Mutating figures in the prose — transposing digits, altering a replicate count — found six distinct ways the checker returned a false pass:

1. it skipped claims it could not parse as numbers, so signed replicate counts were never examined at all;
2. its tolerance was calibrated to value distance, which a digit transposition exceeds, so real typos read as unrelated numbers;
3. it matched substrings without boundaries, so a leniency figure of 0.2% was satisfied by the 70.2% inside an unrelated accuracy;
4. it compared against prose as written, so a claim wrapped across a line break was reported as absent rather than checked;
5. it was not location-aware, so a figure altered in the abstract passed because a correct copy survived in the body — the worst case, since the abstract is the most-read text in a paper;
6. and one load-bearing claim, the human adjudication of §5.3, had no check at all until a mutation of it survived.

Every one of those defects was invisible to reading the code, and every one failed in the direction that accepts a wrong claim. We record this because it is the paper's own subject in miniature. A checker that accepts plausible-but-wrong input is the failure mode we spent §5 measuring in an LLM judge, and the lesson transfers in both directions: a verifier is not established by inspection any more than a judge is, and neither should be trusted on a task it was never adversarially tested against. We do not offer this as a controlled result. It is an experience with a mechanism, and the mechanism is general.

All six defects were subsequently corrected, adversarial mutation tests were rerun, and every numerical claim reported here was re-verified with the corrected checker at commit `83f3738fe2ca9cfb886433e42be103fe87e43fc7` before manuscript freeze.

7.7 Future work

The output-bound diagnostic closes off the tempting low-cost extension on this axis: intermediate bounds mostly produce truncation, not intermediate judging. The next informative extension would vary the judge model or task family, with a new preregistration and adequacy gate, because that would test generality rather than pretend that censored responses form a third operating point.

Beyond that, two directions are scoped but unbuilt. A richer verifiable task family — state-transition tasks with deterministic replay and hash-linked traces — would supply complete verification and an auditable attempt ledger without weakening known truth, and would admit failure modes more substantive than the register-level ones used here. And the diagnostics developed for this apparatus are, we think, more useful to practitioners than the experiments they were built for: a library that helps a team decide whether their adaptive system is still measurable from the data it produces is a different artifact from a reproducible paper, and extracting one from the other is deliberate work rather than a packaging step.

8 Limitations

These are the boundaries of what the work can support. They are stated as a section rather than a closing disclaimer because several of them bound the headline claim directly.

The learner is not an LLM. The adaptive system in §3 is a regularized logistic predictor over engineered features, paired with a selection policy, over synthetic register-transformation tasks. The only real LLM anywhere in this work is the judge in §5. Nothing here is evidence about how LLM agents behave under optimization pressure, and a real tool-using agent is the obvious next rung rather than something this paper reaches.

Capability is static by construction. Ability is a fixed parameter and retraining cannot change it. This is what makes the apparatus interpretable, and it is also why the apparatus cannot show the phenomenon practitioners most fear: narrowing that *masquerades as* genuine capability gain. Demonstrating that requires a setting where capability can actually move, and this one deliberately forbids it.

Two operating points, one judge, one task family, one rubric. §5 reports a tension between accuracy and leniency observed twice. It is not a curve, and we have not shown that no admissible configuration exists — only that we did not find one among those we tested. The completion distribution rules out a clean intermediate output-bound point under the preregistered unresolved-unit rule, but a different judge or task family could behave differently. A single judge model and a single rubric therefore bound the generality further.

The intervention is not decomposable. The reasoning field, the instruction to use it, and the larger output bound changed together. The comparison shows what happened when working space was introduced, but cannot identify which of those three coupled parts carries the effect.

No confirmatory primary was run against the real judge. The preregistered primary was never executed, so no effect is reported, no effect size is estimated, and the parameters that would have governed it — the drift tolerance, the effect margin, the paired test and its replicate count — remain unfrozen. The negative result concerns the feasibility of the measurement, not the absence of the phenomenon.

The apparatus's own validity result is expected. §3.2's effect follows from the construction, and reporting it as though it were a discovery would be circular. It is included because an instrument that failed there could not be trusted in §5, not because it establishes anything about systems in the wild.

Exploitation and narrowing are a taxonomy, not a decomposition. Their sum is an accounting identity. They interact, since a policy can narrow specifically toward evaluator-optimistic regions, so no variance attribution between them is claimed.

Three mechanisms in the wider programme are untouched. Trajectory narrowing — the concentration of an agent's action sequences rather than its task choices — is not meaningful for a predictor-and-policy learner and becomes measurable only with a tool-using agent. Capability formation, where retraining actually changes what the system can do, is excluded by construction here. And world co-adaptation, in which the environment responds to the system's behaviour, is outside the setting entirely. Each is a distinct source of non-identifiability that a complete account would have to address.

The infrastructure finding is about fitness for one purpose. §6 concerns whether a framework can supply archival provenance for a longitudinal experiment. The conditions it fails are ones no regression-testing framework advertises meeting, and nothing in this paper bears on its quality for the job it was built for.

The audit-correction mechanism is inert here. Because capability is flat, the policy-independent reference channel cancels exactly in §3.2's contrast. It is retained for continuity across

the programme and does no work in the reported result.

9 Conclusion

We set out to measure whether a real LLM judge's metric loses its evidentiary value when a system is optimized against it, and built an apparatus capable of distinguishing the two mechanisms by which that could happen. The apparatus works: it detects exposure selection under a known-flat capability floor, well above the minimum effect the design was powered for, and it separates evaluator exploitation from distribution narrowing under a known distortion.

Applied to a real judge, it returned a boundary instead of an effect. The configuration that made the judge inaccurate made it exploitable; the configuration that made it accurate left almost nothing to exploit. A human scoring 50 of 50 on the same items rules out the explanation that the task was ill-posed. At the leniency that survived, the preregistered experiment would have consumed its entire budget on a single replicate pair.

We report no confirmatory effect, and we claim no frontier — two operating points are two points. What we offer is the boundary itself, the apparatus that makes it visible, and an adequacy gate whose two opposed conditions prevent a task from passing by becoming vague. A metric that is accurate enough to trust may be, for that reason, uninformative about the pressure applied to it. Whether that relationship holds beyond the configurations we tested is the obvious next question, and it is cheap to ask.

Provenance for §§3 and 5

Every figure above is re-derived by `tools/verify_claims_ladder.py` from `agent-evaluability/results/*/primary.csv` and the run ledgers, and fails the build if the prose and the artifacts disagree.

Two of its limits are worth knowing when reading its output. It cannot tell a value *mentioned* as an example from one *asserted* as a claim, so §7.6's discussion of a leniency figure satisfies that figure's check on its own. And where two derived values sit within two characters of each other — the screen's and the pilot's leniency — it verifies only that a correct rendering is present, not that every rendering is. Those rows are marked `OK*` in its output.

§	Claim	Ladder row	Evidence
3.2	H2 +0.33834, CI, t(85), 86/86	A1	ag1_primary/primary.csv, Figure 1
3.2	validity demonstration, not discovery	A1 scope	notes/AG1_Result_Report.md §6
3.3	C1 +0.06600, lower bound +0.04645, t(19), 19/20	A2	ag2a_primary/primary.csv, Figure 2
3.3	accounting identity, not a decomposition	B1	notes/AG2a_Result_Report.md
3.3	narrowing lower at $\beta=1$ in 20/20	B2	ag2a_primary/primary.csv
5.1	61.00% accuracy, <code>e11</code> 26.60%	B3	pre-run-20260922T182054Z
5.1	six completion tokens on all 1,075 calls	B3	same ledger
5.1	brevity asymmetry 64.5% vs 26.5%	B3	AG2b pilot audit

§	Claim	Ladder row	Evidence
5.2	99.78% accuracy, e ll 0.20%	B3	pre-run-pilot-20260923T084208Z
5.2	screen replication 99.67% / 0.67%	B3	pre-run-screen-20260923T042425Z
5.2	canary truncation at 384; bound raised to 768	—	<i>AG2b canary failure note</i>
5.3	human 50/50, committed before scoring	B5	<i>AG2b human audit</i> , responses f6be41b
5.4	p 0 0.111%; 42,311 calls per pair	B4	Figure 4
5.5	no admissible point; not a frontier	B3	Figure 3

References

- [1] Allison J. B. Chaney, Brandon M. Stewart, and Barbara E. Engelhardt. How algorithmic confounding in recommendation systems increases homogeneity and decreases utility. In *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys)*, pages 224–232, New York, NY, USA, 2018. Association for Computing Machinery. doi: 10.1145/3240323.3240370. arXiv:1710.11214.
- [2] Guiming Hardy Chen, Shunian Chen, Ziche Liu, Feng Jiang, and Benyou Wang. Humans or llms as the judge? a study on judgement biases. *arXiv preprint arXiv:2402.10669*, 2024. doi: 10.48550/arXiv.2402.10669.
- [3] Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdo Yun, Sungdong Kim, James Thorne, and Minjoon Neubig. Prometheus: Inducing fine-grained evaluation capability in language models. *arXiv preprint arXiv:2310.08491*, 2023. doi: 10.48550/arXiv.2310.08491.
- [4] Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. Prometheus 2: An open source language model specialized in evaluating other language models. *arXiv preprint arXiv:2405.01535*, 2024. doi: 10.48550/arXiv.2405.01535.
- [5] Karl Krauth, Yixin Wang, and Michael I. Jordan. Breaking feedback loops in recommender systems with causal inference. *arXiv preprint arXiv:2207.01616*, 2022. doi: 10.48550/arXiv.2207.01616. Algorithm: CAFL (Causal Adjustment for Feedback Loops).
- [6] Swarnadeep Saha, Xian Li, Marjan Ghazvininejad, Jason E. Weston, and Tianlu Wang. Learning to plan and reason for evaluation with thinking-llm-as-a-judge. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 52565–52583. PMLR, 2025. URL <https://proceedings.mlr.press/v267/saha25b.html>.
- [7] Yuta Saito and Thorsten Joachims. Off-policy evaluation for large action spaces via embeddings. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *PMLR 162*, pages 19089–19122. PMLR, 2022. MIPS.
- [8] Yi Su, Maria Dimakopoulou, Akshay Krishnamurthy, and Miroslav Dudík. Doubly robust off-policy evaluation with shrinkage. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR 119*, pages 9167–9176. PMLR, 2020. arXiv:1907.09623.

- [9] Yu-Xiang Wang, Alekh Agarwal, and Miroslav Dudík. Optimal and adaptive off-policy evaluation in contextual bandits. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR 70*, pages 3589–3597. PMLR, 2017. SWITCH estimator.
- [10] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023. doi: 10.48550/arXiv.2306.05685.